

Лекция 7. Моделирование выполнения параллельных программ

7.1. Проблема Тупика

После того как разработчик определился со способом параллельного выполнения программы (параллельным алгоритмом) необходимо удостовериться, что реализация параллельной программы не приведет к тупикам.

Тупик - ситуация, в которой один или несколько процессов ожидают какого-либо события, которое никогда не произойдет.

Важно отметить, что состояние тупика может наступить не только вследствие логических ошибок, допущенных при разработке параллельных программ, но и в результате возникновения тех или иных событий в вычислительной системе (выход из строя отдельных устройств, нехватка ресурсов и т.п.).

Проблема тупиков имеет многоплановый характер. Это и сложность диагностирования состояния тупика (система выполняет длительные расчеты или "зависла" из-за тупика), и необходимость определенных специальных действий для выхода из тупика, и возможность потери данных при восстановлении системы при устранении тупика.

В данном разделе будет рассмотрен один из аспектов проблемы тупика – анализ причин возникновения тупиковых ситуаций при использовании разделяемых ресурсов и разработка на этой основе методов предотвращения тупиков.

Могут быть выделены следующие **необходимые условия тупика**:

- **условие взаимоисключения** - процессы требуют предоставления им права монопольного управления ресурсами, которые им выделяются ;
- **условие ожидания ресурсов** - процессы удерживают за собой ресурсы, уже выделенные им, ожидая в то же время выделения дополнительных ресурсов;
- **условие неперераспределяемости** - ресурсы нельзя отобрать у процессов, удерживающих их, пока эти ресурсы не будут использованы для завершения работы;
- **условие кругового ожидания** - существует кольцевая цепь процессов, в которой каждый процесс удерживает за собой один или более ресурсов, требующихся следующему процессу цепи

Как результат, для обеспечения отсутствия тупиков необходимо исключить возникновение, по крайней мере, одного из рассмотренных условий.

Далее будет рассматривается модель программы в виде графа "процесс-ресурс", позволяющего обнаруживать ситуации кругового ожидания.

Для этого используют различные теоретические модели для моделирования выполнения параллельных программ. Рассмотрим две модели:

- модель в виде дискретной системы;
- модель в виде сети Петри.

7.2. Модель в виде сети Петри

Другим возможным способом моделирования состояний и функционирования параллельной программы является использование математических моделей и методов исследования дискретных систем, разработанных в рамках теории сетей Петри. При таком подходе в качестве модели программы может быть использована сеть Петри, представляемая размеченным ориентированным графом (V, E, M_0) :

1. Множество вершин сети V разделено на два взаимно пересекающихся подмножества *вершин-переходов* P и *вершин-мест* R

$$P = (p_1, p_2, \dots, p_n), R = (R_1, R_2, \dots, R_m).$$

Вершины-места обычно изображаются кружками, вершины-переходы представляются в виде прямоугольников или линий-барьеров.

2. Граф сети является "двудольным" по отношению к подмножествам вершин P и R , т.е. каждое ребро $e \in E$ соединяет вершину P с вершиной R . Задание ребер графа может быть выполнено, например, при помощи функций инцидентности

$$F: R \times P \rightarrow \{0,1\}, H: P \times R \rightarrow \{0,1\},$$

ненулевые значения которых задают множество ребер E (при $H(p_i, R_j) = 1$ сеть содержит ребро вида $e = (p_i, R_j)$, при $F(R_j, p_i) = 1$ сеть содержит ребро вида $e = (R_j, p_i)$).

3. Разметка формально задается функцией $M: P \rightarrow \{0,1,2,\dots\}$, функция M представляется вектором, в котором i -й компонент задает разметку места R_i . При графическом изображении разметка сети показывается соответствующим числом точек (фишек) внутри кружков-мест.

Максимальное количество фишек которое может вместить место называется **ограниченностью емкости места**.

Начальная разметка сети Петри обозначается M_0 . Например, на рисунке приведена начальная разметка сети Петри с тремя переходами и четырьмя вершинами мест (рис. 7.1) представляется вектором $M_0 = \{1,1,0\}$.

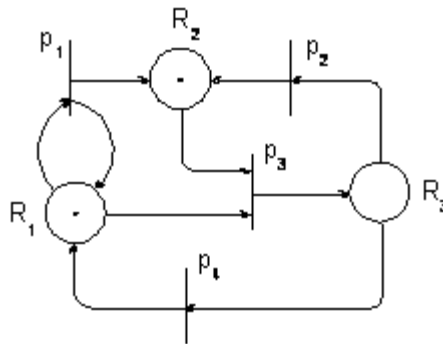


Рис. 7.1. Пример сети Петри

При использовании сетей Петри для описания параллельных программ переходы обычно соответствуют действиям (процессам), а места – условиям (выделению или освобождению ресурсов). Разметка мест в этом случае интерпретируется как количество имеющихся (нераспределенных) единиц ресурса.

Сеть Петри может *функционировать* (изменять свое состояние), переходя от разметки к разметке. Обозначим через $F(R_j)$ множество переходов, к которым имеются ребра из вершины-места R_j

$$F(R_j) = \{p_i \in P: F(R_j, p_i) = 1\};$$

по аналогии, $H(R_j)$ есть множество переходов, из которых имеются ребра в вершину-место R_j

$$H(R_j) = \{p_i \in P: H(p_i, R_j) = 1\}$$

С учетом введенных обозначений правила функционирования сети состоят в следующем:

- Переход p_i может сработать при разметке M только при выполнении условия

$$\forall R_j \in R \Rightarrow M(R_j) - F(R_j, p_i) \geq 0.$$

Данное условие означает, что все входные места перехода p_i содержат хотя бы по одной фишке.

- В результате срабатывания перехода p_i разметка сети M сменяется разметкой M' по правилу:

$$\forall R_j \in R \Rightarrow M'(R_j) = M(R_j) - F(R_j, p_i) + H(p_i, R_j).$$

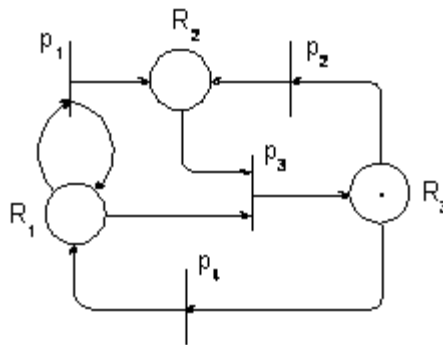


Рис. 7.2. Состояние сети после срабатывания перехода p_3

Другими словами, переход p_i изымает по одной фишке своего входного места и добавляет по одной в каждое свое выходное место. Будем говорить далее, что разметка M' следует за разметкой M , а M предшествует M' , и обозначать этот факт

$$M \xrightarrow{p_i} M'.$$

Так, в сети на рис. 7.1 могут сработать переходы p_1 и p_3 ; состояние сети после срабатывания перехода p_3 показано на рис. 7.2.

Если одновременно может сработать несколько переходов и они не имеют общих входных мест, то их срабатывания могут рассматриваться как независимые действия, выполняемые последовательно или параллельно. Если несколько переходов могут сработать и имеют хотя бы одно общее входное место, то сработать может только один, любой из них.

Сеть Петри, в которой все места 1-ограничены, называется **безопасной**. Безопасная сеть никогда, не допустит, чтобы в переменную было положено новое значение, если старое еще не было использовано по назначению. Нарушения этого правила часто являются причиной ошибок в параллельных программах.

При исследовании процессов функционирования сетей Петри широко используется следующий ряд дополнительных понятий и обозначений:

- разметка M' *достижима* в сети от разметки M

$$M \rightarrow M',$$

если разметка M' может быть получена в результате некоторого количества срабатываний сети, начиная от разметки M ;

- множество разметок, достижимых в порядке срабатывания сети, представляется **разверткой сети**.
- множество достижимых разметок удобно представлять графом достижимости, который показывает все достижимые разметки и последовательности срабатываний переходов, приводящих к ним.

- разметка сети называется *тупиковой*, если при этой разметке ни один из переходов сети не может сработать;
- разметка M *достижима* в сети, если $M_0 \rightarrow M$; множество всех достижимых разметок обозначим через M ;
- переход P_i *достижим* от разметки M , если существует достижимая от M разметка M' , в которой переход P_i может сработать;
- переход P_i *достижим* в сети, если он достижим от M_0 ;
- переход P_i называется *живым*, если он достижим от любой разметки из M ; сеть является *живой*, если все ее переходы живы;
- место R_j называется *ограниченным*, если существует такое число k , что $M(p) \leq k$ для любой разметки из M ; сеть является *ограниченной*, если все ее места ограничены.

В рамках теории сетей Петри разработаны методы, позволяющие для произвольной сети определить [26], является ли сеть ограниченной или живой, проверить достижимость любого перехода или разметки сети. Как результат, данные методы позволяют определить наличие тупиков в сети.